DevOrbit

Functional Requirements, Non-Functional Requirements & Stakeholders

Corrected version • Priorities: Must / Should / Could

1. Stakeholders

Type	Stakeholder / Actor	Role in DevOrbit
Primary	Project Owner / Creator	Creates a project, sets its goal and milestones, invites members, connects GitHub, and marks the project as completed.
Primary	Contributor / Team Member	Joins a project, works on tasks, communicates in channels, views GitHub activity in the Activity Feed, and uses Catch Me Up after being away.
Primary	Project Admin / Manager	Manages project members, roles and project settings; approves join requests; reviews activity, overdue and blocked tasks.
Primary	Platform Admin	Manages platform-level administration, user/access controls and platform configuration, and grants the Moderator role.
Primary	Regular Feed User (logged in)	A registered, authenticated user. Creates public posts, votes, follows users, browses the discovery feed, and can request to join a public project.
Primary	Public Visitor (not logged in)	An unauthenticated visitor with read-only access: can browse public projects, public project pages and the public feed, and can sign up or log in. Following users, voting, posting and requesting to join all require an account.
Primary	Content Moderator	Reviews reported public posts and can remove content, warn a user (via in-app notification), or dismiss a report.
System Actor	AI Project Copilot	Acts on behalf of a user to answer project questions and generate Catch Me Up summaries using permitted project data; it does not act autonomously.
Secondary	Development Team / Squads	Designs, develops, integrates, tests and deploys DevOrbit. DevOps/QA is treated as a specialized responsibility within this team.
Secondary	Course Instructor / Grader	Sets and validates course expectations, reviews the project, and evaluates the implementation and contribution.
External System	GitHub	Provides authorized repository commit, issue and pull-request data through OAuth/signed webhooks.
External Service	LLM API Provider (Claude / OpenAI)	Provides the language-model capability used by Project Copilot and Catch Me Up.
External Service	Vercel / Render / MongoDB Atlas / file storage	Provide hosting and database infrastructure on which the deployed system depends.

2. Functional Requirements (FR)

The requirements below define the user-facing capabilities expected of DevOrbit.

ID	Functional Requirement	Priority
FR-1	The system shall allow a user to sign up and log in securely using an email and password. A successful login shall issue a short-lived access token (15 minutes) and a revocable refresh token (7 days) delivered in an httpOnly cookie.	**Must**
FR-2	The system shall allow a user to create and edit a profile containing a photo, bio and skill tags.	**Must**
FR-3	The system shall allow a user to create a project with a name, description, visibility (public/private), domain, skill tags, starting members and a status (Active, Completed or Archived; new projects start as Active). The Project Owner shall be able to change the project status, including marking the project Completed.	**Must**
FR-4	The Project Owner or Admin shall be able to assign the roles Owner, Admin, Contributor or Viewer to project members.	**Must**
FR-5	The system shall allow users to create and use text channels inside a project with real-time messages and @mentions. Threaded replies are a stretch goal and not required for this requirement to be met.	**Must**
FR-6	The system shall allow a user to send and receive direct messages with another user outside a project channel.	**Should**
FR-7a	The system shall allow a user to create a task with a title, an optional due date and a label, and assign it to a project member.	**Must**
FR-7b	The system shall allow a user to update a task status. The defined statuses are To Do, In Progress, Blocked and Done.	**Must**
FR-7c	The system shall allow one or more tasks to be grouped under a milestone.	**Should**
FR-8	The system shall display project tasks on a Kanban board with one column per status and allow a user to move a task between statuses by dragging it.	**Should**
FR-9	The system shall display one chronological Activity Feed for each project, combining project messages, task changes and connected GitHub events.	**Must**
FR-10	The system shall notify users inside the application about mentions, task assignments and tasks whose due date is within the next 24 hours.	**Should**
FR-11	The Project Owner shall be able to connect a GitHub repository to a project using GitHub OAuth.	**Must**
FR-12	The system shall receive commit, issue and pull-request events from a connected GitHub repository (via signed webhooks) and add them to the project's Activity Feed.	**Must**
FR-13	The system shall allow a user to record a project decision with title, description, participants, date and a link to the related task or discussion.	**Should**
FR-14a	The system shall allow a user to upload a file to a project.	**Should**
FR-14b	The system shall allow a user to create a short Markdown documentation page for a project.	**Could**
FR-15	The system shall provide a project dashboard showing milestone progress, open task counts and upcoming (due within 7 days) and overdue tasks.	**Should**
FR-16a	The system shall allow a user to ask the Project Copilot a free-text question about their project.	**Must**
FR-16b	The Copilot shall ground its answer in that project's permitted messages, tasks, decisions and GitHub activity, rather than relying only on general knowledge.	**Must**
FR-16c	The Copilot shall indicate the project data used to support its answer, where applicable.	**Should**
FR-17	The system shall provide a Catch Me Up function that summarizes what happened in a project since the user's last login.	**Must**
FR-18	The system shall allow users to browse a public list of projects filtered by domain, skill tag or project status.	**Should**
FR-19	The system shall allow a user to search for another user using the skill tags on that user's profile.	**Should**
FR-20	The system shall allow any visitor to view a public project's read-only page, and shall allow a logged-in non-member to request to join it. The Project Owner or Admin shall be able to approve or reject each join request, and the requester shall be notified of the outcome.	**Should**

2. Functional Requirements (FR) — continued

ID	Functional Requirement	Priority
FR-21	The system shall generate a shareable portfolio page for a user listing the projects with status Completed, the user's roles and contribution statistics.	**Could**
FR-22	The system shall provide a project health snapshot showing overdue tasks (past due date and not Done), blocked tasks (status Blocked) and recent activity level.	**Should**
FR-23	The system shall allow a logged-in user to create a public post containing text and optionally one image, independent of a project.	**Must**
FR-24	The system shall allow a logged-in user to upvote or downvote a public post, with at most one active vote per user per post (the user may change or retract it). A post's score is upvotes minus downvotes.	**Must**
FR-25	The system shall display a global feed of public posts ranked by trending score = score / (age_hours + 2)^1.5, where score is the post's net score (upvotes minus downvotes) and age_hours is the time since posting. Ties are broken by newer post first.	**Must**
FR-26	The system shall allow a logged-in user to follow or unfollow another user. A profile shall display the public follower count and the user's total vote score, defined as the sum of the net scores of all of that user's public posts.	**Should**
FR-27	The system shall allow a logged-in user to report a public post. A Moderator shall be able to review the report and take one defined action: remove the post, warn the user, or dismiss the report. A warning shall be delivered to the user as an in-app notification. The Moderator role shall be granted or revoked by a Platform Admin, and the first Platform Admin shall be created by a seed script or environment configuration.	**Should**
FR-28	The system shall allow a logged-in user to log out. Logging out shall revoke the user's refresh token and deny the current access token, so protected resources require a new login afterward.	**Must**
FR-29	The system shall allow a user to add, edit and remove skill tags from the profile.	**Should**
FR-30	The system shall allow users to search public projects by project name or relevant keywords in addition to the defined filters.	**Should**
FR-31	The system shall allow a Project Owner or Admin to invite an existing user to join a project after project creation. The invited user shall be able to accept or decline the invitation.	**Should**
FR-32	The system shall allow an authorized user to view and update task details including title, description, assignee, label, due date, status and milestone.	**Should**
FR-33	The system shall allow a user to view notifications and identify their type, such as mention, assignment, deadline or moderation warning.	**Should**
FR-34	The system shall allow an authorized user to view and update a previously recorded project decision.	**Should**
FR-35	The system shall allow the Project Owner/Admin to view the GitHub connection state and disconnect or re-authorize the connection when permitted.	**Should**
FR-36	If sufficient project data is not available to answer a Copilot question, the system shall clearly state that the available project data is insufficient instead of presenting an unsupported project-specific answer.	**Must**

3. Non-Functional Requirements (NFR)

Quality attributes, security controls, reliability expectations and implementation standards.

ID	Non-Functional Requirement	Priority
NFR-1a	The project Activity Feed and the public feed shall load within 3 seconds under normal classroom-scale use.	**Must**
NFR-1b	Messages sent through a project channel or direct message shall arrive in near real time, with a target of under 1 second under normal classroom-scale use.	**Must**
NFR-2	The system shall support at least 100 concurrent users without noticeable slowdown under class-demo/testing conditions.	**Must**
NFR-3a	Passwords shall be stored using a salted hash such as bcrypt and never in plain text.	**Must**
NFR-3b	Access tokens shall expire after 15 minutes and refresh tokens after 7 days. Refresh tokens shall be stored server-side so they can be revoked, and no token shall remain valid indefinitely.	**Must**
NFR-3c	Repeated failed login attempts for one account shall be throttled or temporarily locked to reduce brute-force risk.	**Should**
NFR-4	Traffic between the browser, backend and database shall use HTTPS/TLS encryption. GitHub access tokens shall additionally be encrypted at rest and never returned to the client or written to logs.	**Must**
NFR-5a	If two users edit the same task or other shared project data concurrently, the system shall detect the change and show that it has changed, rather than silently overwriting one update, so shared data stays consistent. (Merges former NFR-25.)	**Must**
NFR-5b	Dragging a task to a new Kanban status shall update the UI within 300 ms of the drop, even before server confirmation where optimistic UI is used.	**Should**
NFR-6	An in-app notification for a mention, task assignment or deadline shall appear within 5 seconds of the triggering event.	**Should**
NFR-7	The web application shall work on both laptop-sized and phone-sized browser windows without breaking the layout.	**Should**
NFR-8	A file upload shall be limited to a defined maximum size (for example, 10 MB per file), with a clear error message when the limit is exceeded. Uploaded files, profile photos and post images shall be kept in persistent external storage (Cloudinary, S3 or GridFS), not on the hosting platform's local disk, which is wiped on redeploy.	**Should**
NFR-9	If GitHub or the LLM API is slow or temporarily unavailable, core chat, task and feed functionality shall continue to work; only the affected integration or AI functionality may be unavailable. (Merges former NFR-24.)	**Must**
NFR-10a	A Copilot or Catch Me Up response shall return within 10 seconds for a typical project's data volume, or show a visible loading state if it takes longer.	**Should**
NFR-10b	Each Copilot or Catch Me Up request shall remain within a configured maximum context/token budget per request; the configured limit shall be documented before deployment.	**Should**
NFR-11a	The webhook endpoint shall verify the GitHub signature (X-Hub-Signature-256) of each delivery and reject invalid ones. Because GitHub does not automatically redeliver failed webhooks, failed processing of a received event shall be retried with bounded backoff and the final failure logged, and missed events shall be recovered by periodically reconciling with the GitHub API.	**Should**
NFR-11b	Calls to the GitHub API shall respect GitHub rate limits and back off rather than failing hard when a limit is reached.	**Should**
NFR-12	Automatic Portfolio contribution data shall be at most 24 hours stale.	**Could**
NFR-13	The public feed shall enforce one active vote per user per post on the server side so duplicate voting attempts do not corrupt the vote count.	**Must**
NFR-14	A vote update shall be applied as an atomic server-side operation so simultaneous votes cannot corrupt or drop the count.	**Must**
NFR-15	The public-feed trending score shall be recalculated at least every 5 minutes so the ranking does not visibly become stale during normal use or a live demo.	**Should**

3. Non-Functional Requirements (NFR) — continued

ID	Non-Functional Requirement	Priority
NFR-16	The public-feed API shall apply per-user rate limiting to post creation and voting so one user/script cannot flood the feed during a demo.	**Should**
NFR-17	The deployed application shall remain reachable during the demo window; known hosting effects such as a free-tier cold start shall be handled or clearly surfaced.	**Should**
NFR-18	New backend routes shall have at least a basic automated test and shall follow the project's agreed coding conventions; tests and required automated checks shall pass in CI before merging.	**Should**
NFR-19	Protected APIs shall reject unauthenticated requests, requests using an expired, revoked or logged-out session, and requests that exceed the user's assigned project role and permissions. (Merges former NFR-20 and NFR-22.)	**Must**
NFR-21	User-provided inputs shall be validated on the server before being stored or processed.	**Must**
NFR-23	When an operation fails, the system shall provide meaningful error feedback rather than silently failing.	**Should**
NFR-26	Important actions such as creating a post, assigning a task, updating a profile or submitting a report shall give visible success feedback, and any failed operation shall give a meaningful error message rather than failing silently.	**Should**
NFR-27	Equivalent actions across modules shall use consistent terminology and interaction patterns (for example, always 'votes' and 'score', never 'likes').	**Should**
NFR-28	Operations that may take noticeable time shall display an appropriate loading/progress state.	**Should**
NFR-29	The system shall use a modular structure so that changes to authentication, projects, tasks, social feed, GitHub or AI functionality have limited unintended impact on other modules.	**Should**
NFR-30	Backend APIs shall have documented endpoint, request and response behaviour.	**Should**
NFR-31	New backend functionality shall follow the project's agreed coding conventions and pass required automated checks before merge.	**Should**
NFR-32	The architecture shall allow additional project or social features to be added without major restructuring of existing modules.	**Could**

Merged duplicates

NFR-20 and NFR-22 were merged into NFR-19; NFR-24 into NFR-9; NFR-25 into NFR-5a.

4. Out of Scope (Won’t Do)

• Voice or video calls and screen sharing.

• Native iOS/Android apps (a responsive web app only).

• Email, SMS or push notifications (in-app notifications only).

• Git hosting providers other than GitHub (no GitLab or Bitbucket) and only one repository per project.

• Autonomous AI actions: the Copilot only answers questions and summarizes; it never creates or changes data on its own.

• End-to-end encryption of messages.

• Real-time collaborative editing of documentation pages and file version history.

• Payments, subscriptions or paid tiers.

• Third-party social login (Google, Apple) and two-factor authentication.

• A public developer API for third parties.